



WEEK 02

This lab implements a Basic Generative Adversarial Network (GAN) using TensorFlow/Keras to generate handwritten digit images similar to MNIST.

Step 1: Import Libraries

```
In [7]: import os
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist, fashion_mnist
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Flatten, Reshape, LeakyReLU
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import BinaryCrossentropy
import tensorflow as tf
```

Library	Purpose
NumPy	Numerical operations
Matplotlib	Save generated images
Keras	Build neural networks
Adam	Optimizer
BinaryCrossentropy	GAN loss function

Step 2: User Parameters

```
In [3]: dataset_choice = 'mnist'
epochs = 30
batch_size = 128
noise_dim = 100
learning_rate = 0.0002
save_interval = 5
```

```
In [8]: os.makedirs("generated_samples", exist_ok=True)
os.makedirs("final_generated_images", exist_ok=True)
```

Step 3: Dataset Loading & Preprocessing

```
In [4]: (X_train, y_train), _ = mnist.load_data()
X_train = X_train.astype("float32") / 255.0
X_train = np.expand_dims(X_train, axis=-1)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 0s 0us/step

```
In [9]: img_shape = X_train.shape[1:]
```

Step 4: Generator Architecture

```
In [10]: generator = Sequential([
    Dense(256, input_dim=noise_dim),
    LeakyReLU(0.2),
    Dense(512),
    LeakyReLU(0.2),
    Dense(1024),
    LeakyReLU(0.2),
    Dense(np.prod(img_shape), activation='sigmoid'),
    Reshape(img_shape)
])
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Step 5: Discriminator Architecture

```
In [11]: discriminator = Sequential([
    Flatten(input_shape=img_shape),
    Dense(512),
    LeakyReLU(0.2),
    Dense(256),
    LeakyReLU(0.2),
    Dense(1, activation='sigmoid')
])

discriminator.compile(
    optimizer=Adam(learning_rate),
    loss=BinaryCrossentropy(),
    metrics=['accuracy']
)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

Step 6: GAN MODEL

```
In [12]: discriminator.trainable = False

gan = Sequential([generator, discriminator])
gan.compile(
    optimizer=Adam(learning_rate),
    loss=BinaryCrossentropy()
```

```
)
```

```
In [13]: # IMAGE SAVING FUNCTION
def save_generated_images(epoch):
    noise = np.random.normal(0, 1, (25, noise_dim))
    generated_images = generator.predict(noise, verbose=0)

    plt.figure(figsize=(5,5))
    for i in range(25):
        plt.subplot(5,5,i+1)
        plt.imshow(generated_images[i].reshape(28,28), cmap='gray')
        plt.axis('off')

    plt.tight_layout()
    plt.savefig(f"generated_samples/epoch_{epoch:02d}.png")
    plt.close()
```

Step 7: TRAINING LOOP

```
In [14]: half_batch = batch_size // 2

for epoch in range(1, epochs + 1):

    # ----- Train Discriminator -----
    idx = np.random.randint(0, X_train.shape[0], half_batch)
    real_images = X_train[idx]

    noise = np.random.normal(0, 1, (half_batch, noise_dim))
    fake_images = generator.predict(noise, verbose=0)

    real_labels = np.ones((half_batch, 1))
    fake_labels = np.zeros((half_batch, 1))

    d_loss_real = discriminator.train_on_batch(real_images, real_labels)
    d_loss_fake = discriminator.train_on_batch(fake_images, fake_labels)

    d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

    # ----- Train Generator -----
    noise = np.random.normal(0, 1, (batch_size, noise_dim))
    valid_labels = np.ones((batch_size, 1))

    g_loss = gan.train_on_batch(noise, valid_labels)

    # ----- PRINT REQUIRED LOG FORMAT -----
    print(
        f"Epoch {epoch}/{epochs} | "
        f"D_loss: {d_loss[0]:.2f} | "
        f"D_acc: {d_loss[1]*100:.2f}% | "
        f"G_loss: {g_loss:.2f}"
    )

    # ----- SAVE IMAGES PERIODICALLY -----
```

```

    if epoch % save_interval == 0:
        save_generated_images(epoch)

# ----- FINAL IMAGE GENERATION (100 IMAGES) -----
noise = np.random.normal(0, 1, (100, noise_dim))
final_images = generator.predict(noise, verbose=0)

for i in range(100):
    plt.imsave(
        f"final_generated_images/img_{i+1}.png",
        final_images[i].reshape(28,28),
        cmap='gray'
    )

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py:83: UserWarning: The model does not have any trainable weights.
warnings.warn("The model does not have any trainable weights.")

```

```

Epoch 1/30 | D_loss: 0.76 | D_acc: 29.69% | G_loss: 0.92
Epoch 2/30 | D_loss: 0.71 | D_acc: 43.95% | G_loss: 0.87
Epoch 3/30 | D_loss: 0.72 | D_acc: 48.33% | G_loss: 0.82
Epoch 4/30 | D_loss: 0.73 | D_acc: 42.77% | G_loss: 0.78
Epoch 5/30 | D_loss: 0.75 | D_acc: 35.13% | G_loss: 0.73
Epoch 6/30 | D_loss: 0.78 | D_acc: 29.13% | G_loss: 0.69
Epoch 7/30 | D_loss: 0.80 | D_acc: 25.15% | G_loss: 0.65
Epoch 8/30 | D_loss: 0.82 | D_acc: 22.50% | G_loss: 0.62
Epoch 9/30 | D_loss: 0.85 | D_acc: 20.02% | G_loss: 0.58
Epoch 10/30 | D_loss: 0.88 | D_acc: 18.36% | G_loss: 0.55
Epoch 11/30 | D_loss: 0.91 | D_acc: 16.87% | G_loss: 0.52
Epoch 12/30 | D_loss: 0.95 | D_acc: 15.77% | G_loss: 0.48
Epoch 13/30 | D_loss: 0.99 | D_acc: 14.96% | G_loss: 0.46
Epoch 14/30 | D_loss: 1.03 | D_acc: 14.15% | G_loss: 0.43
Epoch 15/30 | D_loss: 1.08 | D_acc: 13.30% | G_loss: 0.41
Epoch 16/30 | D_loss: 1.12 | D_acc: 12.55% | G_loss: 0.39
Epoch 17/30 | D_loss: 1.17 | D_acc: 11.85% | G_loss: 0.37
Epoch 18/30 | D_loss: 1.22 | D_acc: 11.27% | G_loss: 0.35
Epoch 19/30 | D_loss: 1.28 | D_acc: 10.79% | G_loss: 0.33
Epoch 20/30 | D_loss: 1.33 | D_acc: 10.33% | G_loss: 0.31
Epoch 21/30 | D_loss: 1.38 | D_acc: 9.94% | G_loss: 0.30
Epoch 22/30 | D_loss: 1.43 | D_acc: 9.77% | G_loss: 0.29
Epoch 23/30 | D_loss: 1.49 | D_acc: 9.41% | G_loss: 0.28
Epoch 24/30 | D_loss: 1.54 | D_acc: 9.08% | G_loss: 0.26
Epoch 25/30 | D_loss: 1.59 | D_acc: 8.90% | G_loss: 0.25
Epoch 26/30 | D_loss: 1.65 | D_acc: 8.80% | G_loss: 0.24
Epoch 27/30 | D_loss: 1.70 | D_acc: 8.59% | G_loss: 0.24
Epoch 28/30 | D_loss: 1.75 | D_acc: 8.31% | G_loss: 0.23
Epoch 29/30 | D_loss: 1.80 | D_acc: 8.13% | G_loss: 0.22
Epoch 30/30 | D_loss: 1.84 | D_acc: 7.98% | G_loss: 0.21

```

```

In [15]: # ----- FINAL IMAGE GENERATION (100 IMAGES) -----
noise = np.random.normal(0, 1, (100, noise_dim))
final_images = generator.predict(noise, verbose=0)

for i in range(100):
    plt.imsave(

```

```

        f"final_generated_images/img_{i+1}.png",
        final_images[i].reshape(28,28),
        cmap='gray'
    )

```

```

In [16]: # ----- PRE-TRAINED CLASSIFIER (TRANSFER LEARNING) -----
classifier = Sequential([
    Flatten(input_shape=img_shape),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

classifier.fit(X_train, y_train, epochs=3, batch_size=256, verbose=0)

predictions = classifier.predict(final_images)
predicted_labels = np.argmax(predictions, axis=1)

# ----- LABEL DISTRIBUTION OUTPUT -----
print("\nLabel Distribution of Generated Images:")
unique, counts = np.unique(predicted_labels, return_counts=True)
for label, count in zip(unique, counts):
    print(f"Label {label}: {count} images")

```

4/4 ————— 0s 12ms/step

Label Distribution of Generated Images:
 Label 3: 100 images

```

In [17]: import shutil

shutil.make_archive("gan_outputs", 'zip', ".")

```

Out[17]: '/content/gan_outputs.zip'